# Algoritma & Struktur Data

Week 10 - Stack, Queue

# Last Week Review

- Hashing
  - Hashing is used to store the data and hopefully we can retrieve the data in O(1) manner
  - Hashing main problem is Collision, in case the hash function produce Same index
  - There are several way to avoid collision:
    - Make better Hash Function (Ex: Multiply it using prime)
    - Handle the collision instead.
      - Separate Chaining
      - Linear Probing
      - Quadratic Probing
      - Double Hashing

# Outline

- Stack
  - Concept & Usage
  - Implementation
  - Coding
- Queue
  - Concept & Usage
  - Implementation
  - Coding
- QNA

# Stack (Concept)

- Stack is a data structure that have several property
  - The sequence would be LIFO, Last in First Out. Which translate, the last element that enter will be the first one to exit.
  - Imagine a pile of plate. to get the bottom most plate, you need to properly take the top most Plate
- Stack usually have these basic operations
  - Top() = Check the top most element
  - Pop() = Removing the top most element
  - Push() = Insert the element to the top of the stack

Ex Input Data: 10, 50, 30 20, 5

Ex Output Data: 5, 20, 30, 50, 10

| 5 (top) |
| --- |
| 20 |
| 30 |
| 50 |
| 10 (bottom) |

# Stack (Implementation)

- Checking Palindrome
  - Assume input “ABAABA” -> is palindrome
    - Input : ABAABA
    - Output : ABAABA
  - Assume input “BABAA” -> is not palindrome
    - Input : “BABAA”
    - Output : “AABAB”
- Parentheses Checking
  - Assume input “([()])” -> complete parentheses
  - Assume input “(()])” -> invalid parentheses
- Undo Feature
  - If let’s say we move from one page to another. We can push the last url to the stack, so when we click back button we simply pop the last used url

# Stack (Coding)

```
void push(char data, struct element** stack){
    struct element* node = (struct element*)malloc(sizeof(struct element));
    node -> data = data;
    node -> next = *stack;
    (*stack) = node;
}
```

```
void pop(struct element** stack){
    if(*stack != NULL){
        printf("Element popped: %c\n",(*stack) -> data);
        struct element* node = *stack;
        *stack = (*stack) -> next;
        free(node);
    }
    else{
        printf("The stack is empty.\n");
    }
}
```

```
void top(struct element* stack){
    if(stack != NULL){
    printf("Element on top: %c\n", stack -> data);
    }
    else{
        printf("The stack is empty.\n");
    }
}
```

# Queue (Concept)

- Queue is a data structure that have several property
  - The sequence would be FIFO, First in First Out. Which translate, the last element that enter will be the last one to exit.
  - Imagine a line of cinema ticketing. The first one to line up, is the first one to be served
- Queue usually have these basic operations
  - Enqueue() = Push on the back of the queue
  - Dequeue() = Remove the first element of the queue
  - Front() = Get the data of the first element in queue

Ex Input Data: 10, 50, 30, 20, 5

Ex Output Data: 10, 50, 30, 20, 5

| 5 (Last) |
| --- |
| 20 |
| 30 |
| 50 |
| 10 (First) |

# Queue (Implementation)

- Resource Manager
  - In our computer, our processor split the task concurrently based on the queue system.
  - It will serve the process by first come first serve basis.
- Request Queue for API
  - Have you ever fight for a voucher on online shop?
  - The server receive a lot of requests yet it will be queued accordingly, so there will not be a case where one voucher code used by two person.
- Process Sequencer
  - Let’s say we wanted to make a cake, there will be sequence to do so. The queue will ensure the process will be processed accordingly without any glitch or invalid order.

# Queue (Coding)

```c
void enqueue(char data, struct element** queue_tail, struct element** queue_head){
    struct element* node = (struct element*)malloc(sizeof(struct element));
    node -> data = data;
    if(*queue_head != NULL){
        *queue_head = node;
        *queue_tail = node;
    }
    else{
        *queue_tail->next = node
    }
}
```

```c
void dequeue(struct element** queue_head){
    if(*queue_head != NULL){
        printf("Element popped: %d\n",(*queue_head) -> data);
        struct element* node = *queue_head;
        *queue_head = (*queue_head) -> next;
        free(node);
    }
    else{
        printf("The queue is empty.\n");
    }
}
```

```c
void front(struct element* queue_head){
    if(queue_head != NULL){
    printf("Element on top: %d\n", queue_head -> data);
    }
    else{
        printf("The queue is empty.\n");
    }
}
```

# Q N A

# THE END